

NAME : VELMURUGAN A

REG :192521074

1. Introduction

Breadth First Search (BFS) is a graph traversal algorithm used to explore vertices level by level. It is especially useful for finding the shortest path in an unweighted graph because every edge is considered to have equal cost. In this problem, BFS is extended with an additional constraint. A node is considered high-traffic when it has more than 5 neighbors. Such nodes must not be included in the selected path.

The main objective is to find the minimum-hop route from a source node S to a destination node D while respecting the traffic constraint. BFS uses a queue to process nodes in the order in which they are discovered. This guarantees that nodes at distance 1 are processed before nodes at distance 2, and so on.

The degree of a vertex represents the number of edges connected to that vertex. Before adding a neighboring node to the BFS queue, its degree is checked. If its degree is greater than 5, the node is rejected. Otherwise, it is considered for traversal.

This approach is useful in practical network-routing situations where highly connected locations may represent congested intersections, overloaded routers, or busy communication hubs. The constraint can prevent the algorithm from choosing a normally short route that passes through an undesirable node.

Thus, constrained BFS combines shortest-path searching with a feasibility condition. It provides an efficient and easy-to-understand method for solving shortest-path problems where some vertices are restricted.

2. Problem Statement

Given an unweighted graph G, a source vertex S, and a destination vertex D, find a path from S to D having the minimum number of hops. The selected path must not visit any vertex whose number of neighbors is greater than 5.

The problem can be divided into several steps. First, represent the graph using an adjacency list or another suitable graph representation. Second, determine the degree of each vertex by counting its neighbors. Third, identify vertices whose degree is greater than 5. These vertices are treated as high-traffic nodes.

Next, start BFS from S. The source is inserted into a queue and marked as visited. The algorithm repeatedly removes the front vertex from the queue and examines its neighbors. A neighbor is added only when it has not already been visited and its degree is at most 5.

The parent of every newly discovered vertex is stored. This information is required to reconstruct the final route after D is reached. Since BFS explores vertices by increasing distance from S, the first valid visit to D gives the minimum-hop path among the allowed vertices.

If D cannot be reached because every possible route contains a restricted node, the algorithm reports that no valid path exists. Therefore, the problem has two requirements: shortest path and constraint satisfaction.

3. Algorithm and Pseudocode

The proposed algorithm is Constrained Breadth First Search. BFS is selected because the graph is unweighted and the objective is to minimize the number of edges or hops. The additional degree condition is checked whenever a neighbor is considered.

Pseudocode:

BFS_Constrained(G, S, D)

1. Create an empty queue Q.
2. Mark S as visited.
3. Set parent[S] = NULL.
4. Insert S into Q.
5. While Q is not empty:
 6. $u = \text{Remove}(Q)$.
 7. If $u == D$, reconstruct the path and return it.
 8. For every neighbor v of u:
 9. If v is not visited and $\text{degree}(v) \leq 5$:
 10. Mark v as visited.
 11. $\text{parent}[v] = u$.
 12. Insert v into Q.
13. If Q becomes empty, return "No valid path exists".

The degree test prevents high-traffic nodes from entering the search. The visited array prevents repeated processing and avoids unnecessary cycles. The parent array records the predecessor of each discovered node.

Path reconstruction begins at D and repeatedly follows parent pointers until S is reached. The resulting sequence is reversed to obtain the route from S to D.

One implementation detail is that S should also satisfy the constraint if the problem requires every visited node, including the source, to have at most 5 neighbors. Similarly, D should satisfy the constraint because it is visited by the path.

4. Working Principle and Step-by-Step Execution

The working principle of constrained BFS is based on level-order graph exploration. Initially, only the source S is in the queue. BFS then explores all valid vertices one hop away from S. After that, it explores valid vertices two hops away, followed by three-hop vertices, and so on.

Step 1: Calculate or obtain the degree of every vertex.

Step 2: Mark every vertex with degree greater than 5 as restricted.

Step 3: Put S into the queue and mark it visited.

Step 4: Remove the front vertex from the queue.

Step 5: Examine each neighbor of the removed vertex.

Step 6: Ignore a neighbor if it has already been visited.

Step 7: Ignore a neighbor if its degree is greater than 5.

Step 8: Add every remaining valid neighbor to the queue and store its parent.

Step 9: Continue until D is reached or the queue becomes empty.

Step 10: Reconstruct the shortest valid route using the parent array.

For example, consider a route S-A-B-D. If B has six neighbors, B is restricted and this route cannot be selected. If another route S-C-E-D contains only allowed vertices, BFS can select it if it is the shortest valid route.

The important point is that ordinary BFS alone is not enough. The degree condition must be applied during traversal. This transforms the original graph into an effective graph containing only permitted vertices, after which BFS finds the shortest route.

5. Correctness and Complexity Analysis

The algorithm is correct because BFS processes vertices in non decreasing order of their distance from S. Therefore, when a valid destination D is first discovered, no shorter valid path to D can exist.

The constraint is also correctly enforced. A vertex is inserted into the queue only when its degree is less than or equal to 5. Hence, no restricted vertex is expanded or used as an intermediate vertex. If the source is required to satisfy the constraint, it must be checked before BFS starts.

The parent array guarantees that the discovered path can be reconstructed. For every visited vertex v , $\text{parent}[v]$ identifies the vertex from which v was first reached. Following these parent relationships from D to S gives the exact BFS path.

For complexity analysis, assume the graph contains V vertices and E edges. With an adjacency-list representation, BFS visits each allowed vertex at most once and examines each relevant edge at most a constant number of times. Therefore, the time complexity is $O(V + E)$.

The visited array, parent array, degree information, and BFS queue require $O(V)$ additional space. Thus, the space complexity is $O(V)$, excluding the memory required to store the input graph.

The algorithm is efficient for large sparse graphs and is particularly suitable when the graph has many vertices but relatively few edges.

6. Applications, Advantages and Limitations

Constrained BFS can be applied to many real-world routing and network problems. In transportation networks, highly connected intersections can represent congested locations. In computer networks, nodes with many connections may represent overloaded routers or network hubs. In social networks, highly connected users can be excluded when a route should avoid heavily connected accounts.

Advantages include simplicity, guaranteed minimum-hop routing in unweighted graphs, and direct enforcement of the node-degree constraint. The algorithm also avoids unnecessary exploration of restricted vertices. Its $O(V + E)$ time complexity makes it practical for many graph-processing tasks.

Another advantage is flexibility. The threshold can be changed from 5 to another value depending on the application. Additional restrictions can also be added, such as blocked nodes or unavailable links.

However, there are limitations. The method optimizes the number of hops, not physical distance, travel time, bandwidth, or monetary cost. A route with fewer hops may still be slower in a real network. Also, excluding high-degree nodes may remove all possible routes between S and D.

The algorithm assumes that the graph is unweighted. For weighted graphs, algorithms such as Dijkstra's algorithm may be more appropriate when edge weights are nonnegative. If constraints depend on changing traffic conditions, the graph and degree information may also need to be updated dynamically.

7. Conclusion

Constrained BFS provides an effective solution for finding the shortest path between a source S and destination D in an unweighted graph while avoiding high-traffic nodes. A high-traffic node is defined as a vertex having more than 5 neighbors.

The algorithm combines the level-order exploration property of BFS with a degree-based filtering condition. Every candidate neighbor is checked before it is added to the queue. This ensures that restricted nodes are not included in the resulting route.

Because BFS explores paths according to increasing hop count, the first valid path reaching D is guaranteed to have the minimum number of hops among all paths satisfying the constraint. The parent array allows the exact route to be reconstructed.

The method has $O(V + E)$ time complexity and $O(V)$ auxiliary space when an adjacency-list representation is used. These characteristics make it suitable for large unweighted graphs.

The approach is useful for traffic-aware routing, network management, communication systems, and other applications where highly connected nodes should be avoided. Its main limitation is that minimum hops do not necessarily represent minimum real-world cost or time.

Overall, constrained BFS demonstrates how a standard graph algorithm can be modified to handle practical restrictions without losing its shortest-path property. The same principle can be extended to other constraints and graph-search problems.